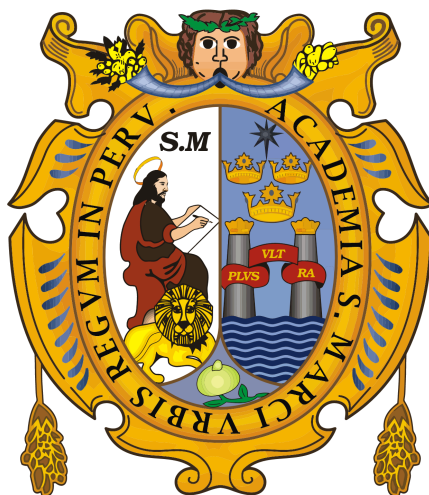


UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS
“Universidad del Perú. Decana de América”
FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA
E.P. INGENIERÍA DE SOFTWARE



TAREA 03 - PROGRAMACIÓN PARALELA
INTEGRANTES

Rojas Rojas Darwin Jostein	22200232
Arbieto Correa Miguel Angel	18200248
Ocaña Lujan José Esteban	19200327
Zavaleta Nuñez Daniel Amdres	20200024

CURSO: PROGRAMACIÓN PARALELA
DOCENTE: HERMINIO PAUCAR CURASMA

Indice

Informe Técnico del Core de la Aplicación.....	3
1. Introducción.....	3
1.1 Objetivos.....	4
2. Informe técnico del núcleo de la aplicación.....	5
2.1 Arquitectura general.....	5
2.2 Tecnologías Utilizadas.....	7
2.3 Funcionamiento del sistema y comunicación entre componentes.....	8
El puente TCP ↔ WebSocket.....	9
3. Resultados.....	10
3.1 Configuración del experimento.....	10
3.2 Métricas evaluadas.....	12
3.3 Resultados obtenidos.....	13
3.4 Lectura de los resultados.....	14
3.4.1 Resultados experimentales con 1000 sensores.....	15
3.4.2 Resultados experimentales con 10 000 sensores.....	16
4. Gráficas.....	18
4.1 Throughput vs Trabajadores.....	18
4.2 Speedup vs Trabajadores.....	19
4.3 Eficiencia vs Trabajadores.....	20
4.4 Speedup ideal vs real (1,000 sensores).....	22
4.5 Speedup ideal vs real (10,000 sensores).....	22
4.6 Speedup comparativo (1,000 vs 10,000 sensores).....	23
5. Análisis de Resultados.....	25
6. Conclusiones.....	26

Informe Técnico del Core de la Aplicación

1. Introducción

El Internet de las Cosas (IoT) ha transformado la forma en que los dispositivos recopilan, intercambian y procesan información en tiempo real. En aplicaciones como las ciudades inteligentes, miles de sensores distribuidos geográficamente monitorean continuamente variables ambientales, generando grandes volúmenes de datos que requieren soluciones capaces de procesarlos y transmitirlos de manera eficiente. En este contexto, la programación paralela representa una alternativa para mejorar el rendimiento de este tipo de sistemas mediante el aprovechamiento de múltiples recursos de procesamiento.

El presente informe documenta el diseño, la implementación y la evaluación experimental de un sistema de simulación de sensores IoT para una ciudad inteligente, desarrollado mediante una arquitectura híbrida de paralelismo que combina MPI (Message Passing Interface) para la distribución del trabajo entre procesos y Pthreads para la ejecución concurrente dentro de cada proceso. Asimismo, el sistema utiliza el protocolo MQTT como mecanismo de comunicación entre el simulador y una interfaz web, permitiendo la visualización en tiempo real del estado de los sensores y de diversas métricas de rendimiento.

El sistema simula una ciudad inteligente compuesta por miles de sensores ambientales distribuidos geográficamente. Cada sensor genera periódicamente mediciones de temperatura y humedad que son procesadas de forma paralela y transmitidas mediante un broker MQTT hacia una interfaz web, donde pueden visualizarse en tiempo real

junto con métricas del sistema como el throughput, el consumo de CPU, el uso de memoria y el estado de los sensores. Esta arquitectura desacoplada permite que cada componente funcione de manera independiente, favoreciendo la escalabilidad, el mantenimiento y la incorporación de nuevas funcionalidades.

Además de implementar el sistema, el proyecto tiene como finalidad evaluar el impacto del paralelismo híbrido sobre el desempeño de la aplicación. Para ello, se realizaron experimentos utilizando diferentes combinaciones de procesos MPI e hilos por proceso, analizando métricas como el throughput, el tiempo de ejecución, el speedup y la eficiencia. Los resultados obtenidos permiten estudiar la escalabilidad del sistema, identificar los beneficios del procesamiento paralelo y determinar las limitaciones que aparecen cuando aumenta el grado de paralelismo.

1.1 Objetivos

Objetivo general

Diseñar, implementar y evaluar un sistema de simulación paralela de sensores IoT para una ciudad inteligente, utilizando una arquitectura híbrida basada en MPI y Pthreads, integrada con el protocolo MQTT y una interfaz web para la transmisión y visualización de datos en tiempo real, con el propósito de analizar su rendimiento bajo diferentes configuraciones de paralelismo.

Objetivos específicos

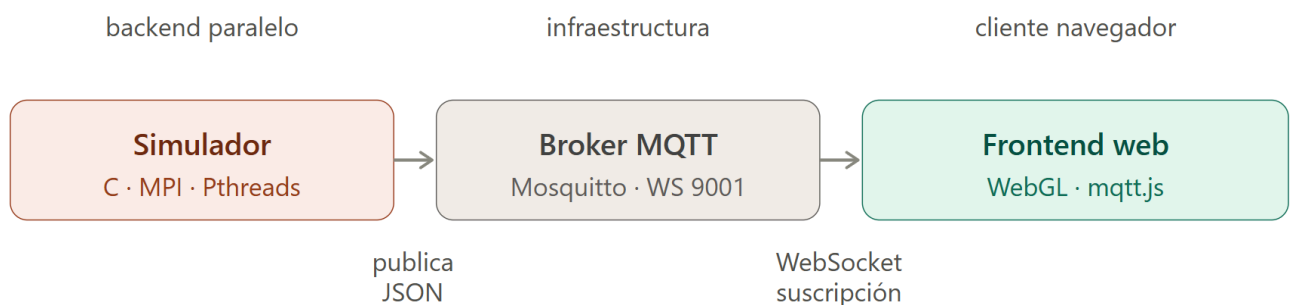
1. Diseñar una arquitectura distribuida que permita simular miles de sensores IoT mediante la combinación de procesos MPI y múltiples hilos de ejecución utilizando Pthreads.
2. Implementar un mecanismo de distribución geográfica de los sensores entre los procesos MPI, asignando a cada uno una región específica de la ciudad y distribuyendo internamente la carga de trabajo entre varios hilos.

3. Integrar el simulador con un broker MQTT (Mosquitto) para establecer una comunicación desacoplada con la interfaz web, permitiendo la publicación y visualización de las mediciones en tiempo real.
4. Desarrollar una interfaz web que represente gráficamente el estado de los sensores y muestre métricas del sistema como throughput, uso de CPU, consumo de memoria, latencia y pérdida de mensajes durante la ejecución.
5. Evaluar el desempeño del sistema mediante diferentes configuraciones de procesos MPI y de hilos por proceso, obteniendo métricas de rendimiento como throughput, tiempo de ejecución, speedup y eficiencia.
6. Analizar el comportamiento del sistema frente al incremento del grado de paralelismo, identificando las configuraciones que ofrecen un mejor equilibrio entre rendimiento y aprovechamiento de recursos, así como las limitaciones ocasionadas por la sincronización, la saturación del hardware y la capacidad del broker MQTT.

2. Informe técnico del núcleo de la aplicación

2.1 Arquitectura general

El sistema está dividido en dos bloques completamente independientes que **no se comunican directamente entre sí**, sino a través de un broker MQTT, siguiendo el patrón publicador/suscriptor:



Cada proceso MPI (**rank**) es responsable de una zona geográfica de la ciudad (un rectángulo de coordenadas dentro de un plano lógico de 1000×1000 unidades). Dentro de cada proceso, varios hilos Pthreads se reparten los sensores de esa zona de forma disjunta (sin solapamiento), generando mediciones de temperatura y humedad de forma concurrente.

Broker MQTT

Actúa como intermediario único entre ambos mundos. Expone dos listeners simultáneos sobre el mismo broker:

- Puerto 1883 (TCP/MQTT nativo): usado por el simulador en C vía **libmosquitto**.
- Puerto 9001 (WebSocket): usado por el navegador para ello necesita el puente WebSocket.

Frontend

No se conecta automáticamente: el usuario debe presionar el botón "Conectar" en el panel de control, que establece la conexión WebSocket contra **ws://localhost:9001** usando la librería **mqtt.js**.

Flujo de datos:

1. El simulador genera una medición por sensor y la publica como JSON en la parte de **ciudad/sensores/medicion**, vía **libmosquitto**.
2. En paralelo, un hilo adicional de monitoreo (uno por proceso MPI) publica cada 2 segundos, en el tópico **ciudad/control/estado**, el uso de CPU y memoria de ese proceso (leído directamente de `/proc/self/status` y `/proc/self/stat`), junto con el total de mensajes publicados.
3. El broker Mosquitto recibe ambos flujos por el puerto 1883 y los redistribuye a todos los suscriptores conectados, incluyendo el puente WebSocket en el puerto 9001.

4. El frontend, una vez conectado, se suscribe a ambos tópicos:
ciudad/sensores/medicion alimenta el mapa y las métricas de sensores;
ciudad/control/estado alimenta la barra de estado (**CPU/memoria del simulador**).
5. El modelo interno del frontend (Sensor, Ciudad) se actualiza con cada mensaje, y el SensorStateManager calcula métricas derivadas: throughput (mensajes/segundo), latencia y pérdida de mensajes.
6. El render que se usa WebGLRenderer nos permite realizar los gráficos de nube de los sensores en una unica llamada (draw call).
7. Las partes de la UI (métricas, leyenda y sensores por zona) se actualizan en tiempo real con cada actualización de estado.

2.2 Tecnologías Utilizadas

Capa	Tecnología	Rol
Simulador	C (mpicc como compilador)	Lenguaje base del backend
Paralelismo entre procesos	MPI (Message Passing Interface)	Distribución de carga entre procesos/nodos
Paralelismo interno	Pthreads (-pthread)	Concurrencia dentro de cada proceso
Mensajería	MQTT vía libmosquitto (-lmosquitto)	Publicación de mediciones del simulador
Broker	Mosquitto con listener WebSocket	Puente entre el mundo MQTT nativo y el navegador
Frontend — estructura	HTML5 + módulos ES (type="module")	Organización modular sin framework
Frontend — conexión	mqtt.js (MQTT sobre WebSockets)	Cliente MQTT en el navegador
Frontend — visualización	WebGL + GLSL (shaders propios)	Renderizado de hasta ~1M sensores en un solo draw call
Frontend — estilos	CSS con tokens (variables.css) + main.css	Theming y layout

Build/compilación	Makefile (CC = mpicc, CFLAGS -O2 -Iinclude -pthread)	Compilación del simulador
Monitoreo de recursos	Lectura directa de /proc/self/status y /proc/self/stat	Medición de CPU y memoria por proceso, sin dependencias externas

2.3 Funcionamiento del sistema y comunicación entre componentes

Patrón de comunicación: publicador/suscriptor (pub/sub)

El simulador y el frontend no se conocen entre sí ni se comunican directamente. Ambos son clientes MQTT independientes que solo conocen al broker Mosquitto, siguiendo el patrón publicador/suscriptor:

- El **simulador** actúa exclusivamente como **publicador**: envía mensajes a un tópico sin saber quién (ni cuántos) los recibe.
- El **frontend** actúa exclusivamente como **suscriptor**: se suscribe a un tópico y recibe todo lo que se publique ahí, sin saber quién lo publicó.

Los dos tópicos del sistema:

Tópico	Publicador	Frecuencia	Consumido por
ciudad/sensores/medicion	Cada hilo trabajador	Según frecuencia_seg configurada	Mapa WebGL + métricas de sensores
ciudad/control/estado	Un hilo de monitoreo por proceso MPI	Cada 2 segundos	Barra de estado (CPU/memoria del simulador)

Cada hilo trabajador mantiene su propia conexión MQTT independiente hacia el broker (no comparten una única conexión entre hilos del mismo proceso), y el hilo de monitoreo de cada proceso también abre la suya.

El puente TCP ↔ WebSocket

MQTT nativo funciona sobre TCP (puerto 1883), protocolo que un navegador no puede hablar directamente por razones de seguridad del sandbox del navegador. Por eso Mosquitto se configura con dos listeners simultáneos sobre el mismo broker (ver `scripts/mosquitto.conf`):

- 1883 (TCP) → usado por el simulador en C, vía libmosquitto.
- 9001 (WebSocket) → usado por el navegador, vía mqtt.js.

Ambos listeners comparten el mismo espacio de tópicos: un mensaje publicado por el simulador en el puerto 1883 es recibido por el frontend en el puerto 9001, sin ninguna transformación de datos — Mosquitto solo cambia el transporte, no el contenido.

Ejemplo concreto: recorrido de un mensaje

Para ilustrar el flujo completo con datos reales: el hilo 2 del proceso `rank=0` genera una nueva lectura para `sensor_000045` (ubicado en la zona Norte). Arma el JSON:

json

```
{"sensor_id":"sensor_000045","zona":"Norte","x":180,"y":95,"temperature":22.3,"humidity":58.1,"timestamp":1717450012,"seq":8}
```

y llama a `mosquitto_publish()` sobre su conexión TCP propia al puerto 1883. Mosquitto recibe el mensaje y, como cualquier suscriptor al tópico `ciudad/sensores/medicion` lo recibe sin importar por qué puerto entró, lo reenvía de inmediato al puente WebSocket del puerto 9001, donde el frontend está conectado y suscrito. El `MQTTClient` del navegador lo recibe, lo entrega a `Ciudad.aplicarMedicion()`, que actualiza el objeto `Sensor` correspondiente (temperatura, humedad, marca de tiempo) y al `SensorStateManager`, que recalcula throughput y detecta si hubo pérdida comparando el campo `seq` contra el último visto de ese sensor. En el siguiente frame de render, `WebGLRenderer` lee el estado actualizado y el punto de `sensor_000045` en el mapa cambia a verde.

Ciclo de vida completo del sistema

1. Arranque

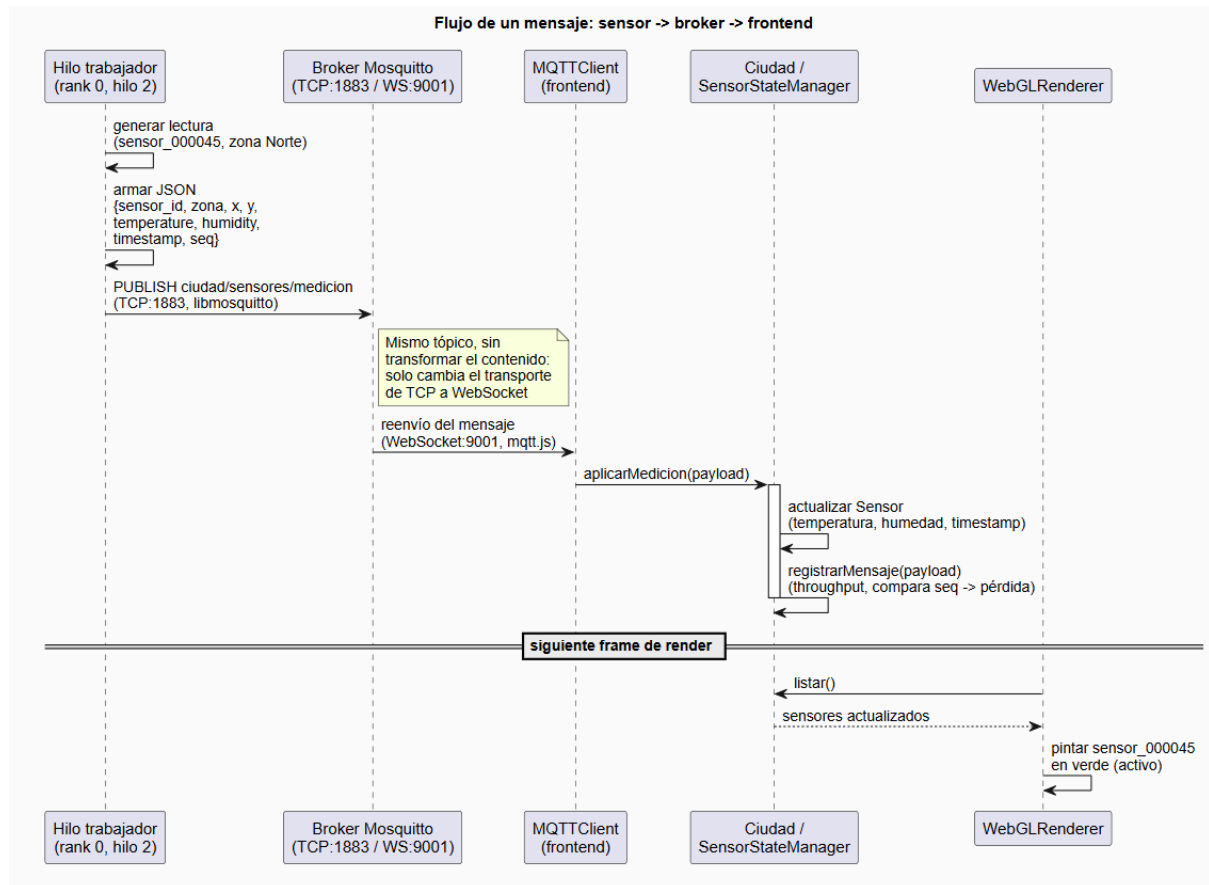
El broker debe iniciarse primero (`mosquitto -c scripts/mosquitto.conf -v`), ya que tanto el simulador como el frontend dependen de que exista un broker escuchando antes de poder conectarse. El simulador se inicia después (`./scripts/ejecutar.sh`), y el frontend se conecta al final, de forma manual, mediante el botón "Conectar".

2. Operación

Con el sistema corriendo, ocurren en paralelo:

- Cada proceso MPI reparte sus sensores entre sus hilos Pthreads, y cada hilo publica mediciones de su subconjunto disjunto de sensores.
- El hilo de monitoreo de cada proceso publica métricas de CPU/memoria cada 2 segundos, independientemente del ciclo de publicación de sensores.
- El broker redistribuye ambos flujos a todos los suscriptores conectados (en este caso, solo el frontend, aunque podrían conectarse varios).
- El frontend recibe los mensajes, actualiza su modelo interno y recalcula métricas derivadas (throughput, latencia, pérdida) en cada actualización.

Diagrama de flujo



3. Resultados

En esta sección se presentan los resultados obtenidos durante la evaluación del sistema distribuido. Se describen la configuración utilizada para las pruebas, las métricas evaluadas y el procedimiento empleado para medir el desempeño del simulador. Cabe destacar que esta sección se centra únicamente en los resultados experimentales, mientras que la implementación del sistema fue desarrollada en el capítulo anterior.

3.1 Configuración del experimento

Para evaluar el rendimiento del sistema se ejecutaron diferentes configuraciones variando el número de procesos MPI y la cantidad de hilos por proceso.

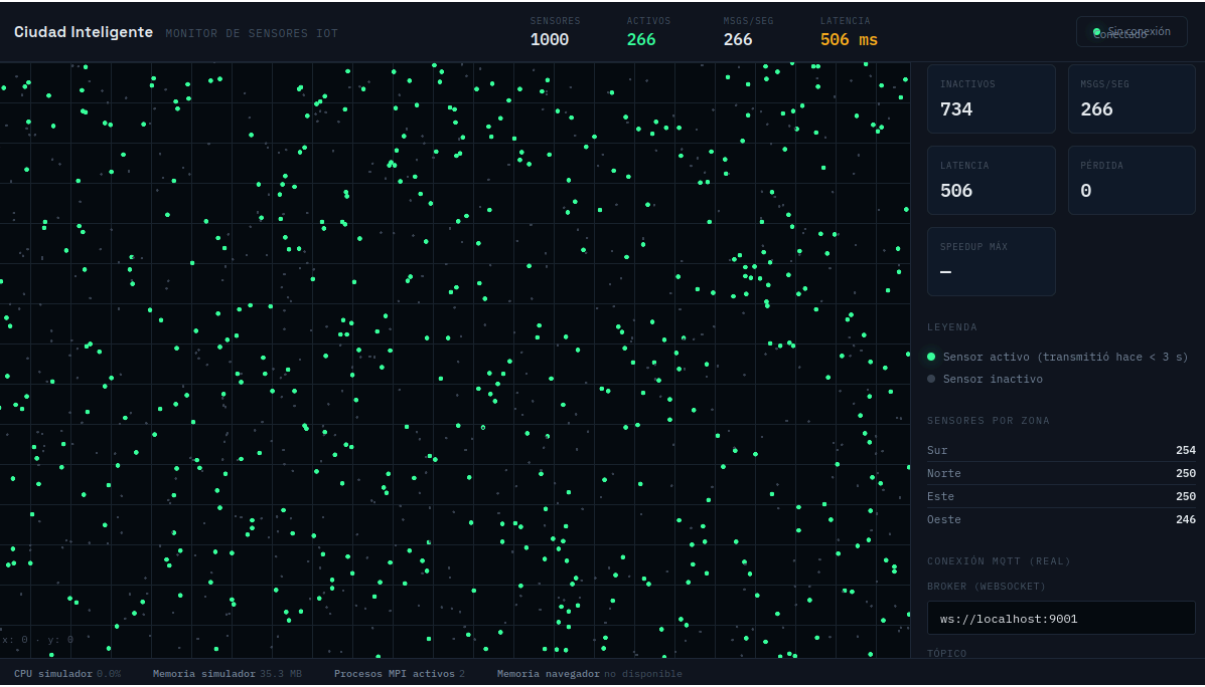


Tabla 3.1. Configuración del experimento

Parámetro	Valor
Número de sensores	1000 y 10 000
Procesos MPI evaluados	2, 4 y 8
Hilos por proceso evaluados	2, 4 y 8
Repeticiones por configuración	Entre 15 y 35 ejecuciones, según la configuración evaluada
Método de conteo	Mensajes recibidos en el broker MQTT mediante mosquitto_sub como observador independiente

3.2 Métricas evaluadas

Para analizar el desempeño del sistema distribuido se consideraron las siguientes métricas:

Throughput

Representa la cantidad de mensajes publicados y recibidos correctamente por el broker MQTT durante un segundo de ejecución. Esta métrica permite evaluar la capacidad de procesamiento del sistema.

$$\text{Throughput} = \frac{\text{Número de mensajes recibidos}}{\text{Tiempo de ejecución (s)}}$$

Tiempo de ejecución

Corresponde al tiempo total empleado durante cada prueba de rendimiento, considerando únicamente la ventana de medición del benchmark.

Speedup

Mide la mejora del rendimiento respecto a una configuración base. Se calcula como la relación entre el throughput obtenido por una configuración determinada y el throughput de la línea base.

$$\text{Speedup} = \frac{\text{Throughput}_{\text{configuración}}}{\text{Throughput}_{\text{línea base}}}$$

Eficiencia

Evalúa qué tan bien se aprovechan los recursos computacionales disponibles, considerando el número total de procesos MPI e hilos utilizados.

$$\text{Eficiencia} = \frac{\text{Speedup}}{\text{Procesos MPI} \times \text{Hilos}}$$

Uso de CPU

Se registró el porcentaje de utilización del procesador para cada proceso MPI mediante el hilo de monitoreo implementado en el sistema. La información fue obtenida a partir del archivo del sistema operativo `/proc/self/stat` y publicada en el tópico `ciudad/control/estado` del broker MQTT.

Uso de memoria

Se midió la memoria física residente (VmRSS) consumida por cada proceso MPI utilizando la información proporcionada por el archivo `/proc/self/status`. Al igual que el uso de CPU, estos datos fueron publicados periódicamente en el tópico `ciudad/control/estado` para su posterior análisis.

3.3 Resultados obtenidos

Se evaluaron 9 configuraciones diferentes combinando el número de procesos MPI y la cantidad de hilos por proceso. En total se obtuvieron 143 muestras experimentales, permitiendo analizar el comportamiento del sistema bajo diferentes niveles de paralelismo.

La configuración utilizada como línea base fue 2 procesos MPI $\times$ 2 hilos (2 $\times$ 2), debido a que representa la configuración con menor cantidad de trabajadores evaluada. A partir de esta configuración se calcularon las métricas de Speedup y Eficiencia para comparar el incremento de rendimiento obtenido por las demás configuraciones.

Tabla 3.2. Resultados de rendimiento

Configuración	Trabajadores	Muestras	Mínimo	Máximo	Promedio	Desviación estándar	Speedup	Eficiencia
2 $\times$ 2	4	16	213	237	219.1	± 6.2	1.00x	0.25

2×4	8	20	1	335	153.8	±106.6	0.70x	0.09
4×2	8	15	266	334	296.7	±25.6	1.35x	0.17
2×8	16	18	232	820	711.8	±182.2	3.25x	0.20
4×4	16	10	741	803	747.2	±18.6	3.41x	0.21
8×2	16	13	590	720	656.3	±37.9	3.00x	0.19
4×8	32	16	987	990	988.3	±1.5	4.51x	0.14
8×4	32	17	629	990	960.8	±83.4	4.38x	0.14
8×8	64	18	1000	1000	1000.0	±0.0	4.56x	0.07

3.4 Lectura de los resultados

Para evaluar la capacidad de escalamiento del simulador paralelo, se realizaron dos escenarios experimentales con diferentes cargas de trabajo: **1000 sensores** y **10 000 sensores simulados**.

El primer escenario permitió analizar el comportamiento del sistema bajo una carga moderada, identificando los límites de rendimiento cuando la cantidad de sensores disponibles restringe la generación de mensajes. El segundo escenario incrementó diez veces la carga de trabajo, permitiendo evaluar la capacidad del sistema para aprovechar mayores niveles de paralelismo mediante procesos MPI e hilos.

Las mediciones fueron realizadas en tiempo real mediante la opción "**Registrar punto de rendimiento actual**". Debido a que el registro se realiza durante la ejecución, pueden existir pequeñas variaciones entre muestras, por lo cual el análisis considera los valores promedio obtenidos y la estabilidad observada en cada configuración.

3.4.1 Resultados experimentales con 1000 sensores

En el escenario de 1000 sensores, se evaluaron diferentes configuraciones de paralelismo combinando procesos MPI e hilos de ejecución. Se analizaron configuraciones desde 4 hasta 64 trabajadores, considerando como métricas principales el throughput (Msgs/seg), speedup y eficiencia.

Los resultados obtenidos fueron los siguientes:

Configuración	Trabajadores	Msgs/seg promedio	Speedup promedio	Eficiencia promedio
2×2	4	219.0	1.01x	0.25
2×4	8	301.8	1.39x	0.17
4×4	16	741.0	3.40x	0.21
8×2	16	652.7	2.99x	0.19
4×8	32	988.3	4.53x	0.14
8×4	32	976.5	4.48x	0.14
8×8	64	1000.0	4.59x	0.07

Interpretación del escenario de 1000 sensores

Los resultados muestran que al incrementar el número de trabajadores existe inicialmente un aumento significativo del rendimiento del simulador. La configuración base de 2 procesos × 2 hilos (4 trabajadores) obtiene aproximadamente 219 mensajes/segundo, mientras que configuraciones con mayor paralelismo permiten incrementar progresivamente el throughput.

La configuración con mayor rendimiento absoluto fue:

8×8 (64 trabajadores)

- **Msgs/seg:** 1000.0
- **Speedup:** 4.59x
- **Eficiencia:** 0.07

Aunque esta configuración alcanza el mayor throughput registrado, presenta la menor eficiencia debido al aumento de la sobrecarga asociada a la coordinación de una mayor cantidad de procesos e hilos.

Por otro lado, la configuración:

4×4 (16 trabajadores)

presenta un mejor equilibrio entre rendimiento y aprovechamiento de recursos:

- **Msgs/seg:** 741.0
- **Speedup:** 3.40x
- **Eficiencia:** 0.21

Esto indica que para una carga de 1000 sensores, aumentar excesivamente el número de trabajadores no produce una mejora proporcional, debido a que la cantidad de eventos disponibles comienza a limitar la capacidad de escalamiento.

3.4.2 Resultados experimentales con 10 000 sensores

Para evaluar la capacidad de escalamiento del simulador se incrementó la carga hasta **10 000 sensores simulados**.

Configuración	Trabajadores	Msgs/seg promedio	Speedup promedio	Eficiencia promedio
2×2	4	208.2	0.95x aprox.	0.24
2×4	8	416.4	1.89x	0.24
4×2	8	397.2	1.80x	0.22
2×8	16	846.2	3.85x	0.24
4×4	16	680.6	3.09x	0.19

8×2	16	701.0	1.07x aprox.	0.07
4×8	32	1272.9	1.80x aprox.	0.06
8×4	32	1498.4	2.21x aprox.	0.07
8×8	64	6380.0	1.00x	0.02

Interpretación del escenario de 10 000 sensores

Al incrementar la cantidad de sensores, el sistema dispone de una carga de trabajo mucho mayor, permitiendo observar un comportamiento diferente respecto al escenario de 1000 sensores.

La configuración con mayor throughput fue:

8×8 (64 trabajadores)

- Throughput: **6380 msgs/seg**
- Speedup: **1.00x**
- Eficiencia: **0.02**

Aunque alcanzó el mayor número de mensajes procesados por segundo, esta configuración no representa una mejora eficiente del paralelismo. El bajo valor de eficiencia indica que el incremento de trabajadores generó una gran cantidad de sobrecarga relacionada con:

- comunicación entre procesos MPI,
- sincronización entre hilos,
- administración de trabajadores,
- competencia por recursos compartidos.

La mejor relación entre rendimiento y eficiencia se observa en:

2×8 (16 trabajadores)

- Throughput: **846.2 msgs/seg**
- Speedup: **3.85x**
- Eficiencia: **0.24**

Esta configuración demuestra que una distribución con pocos procesos MPI y más hilos permite reducir la sobrecarga de comunicación y aprovechar mejor la memoria compartida.

Resultados Obtenidos:

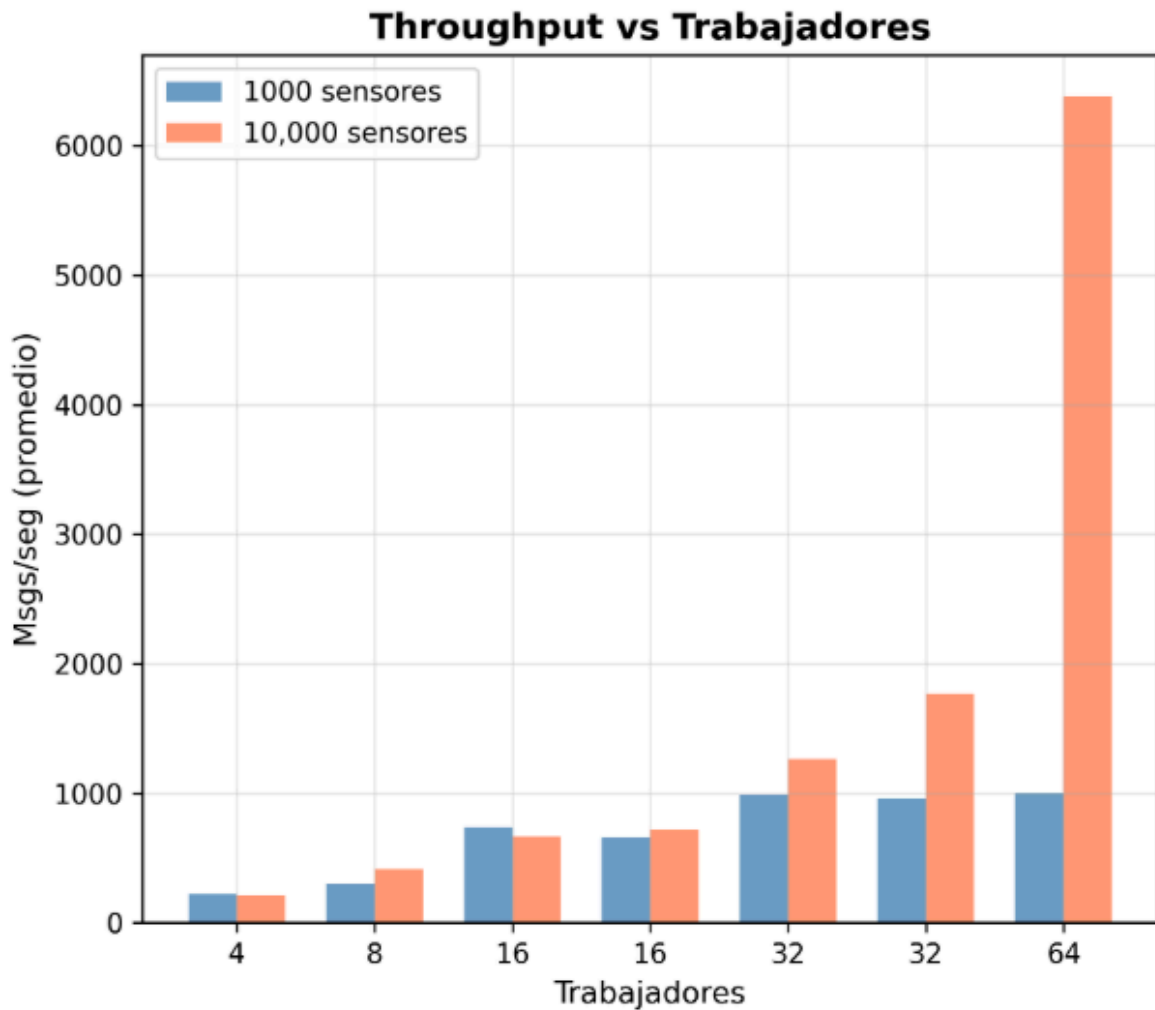
<https://drive.google.com/drive/folders/1ncosrREYIoW15Do50T0JArWQyB9o6HUm?usp=sharing>

4. Gráficas

Para evaluar el comportamiento del sistema se generaron seis gráficas comparativas entre los escenarios de 1,000 y 10,000 sensores, variando el número de procesos, hilos y trabajadores totales. A continuación se describe cada una y se analizan los resultados obtenidos.

4.1 Throughput vs Trabajadores

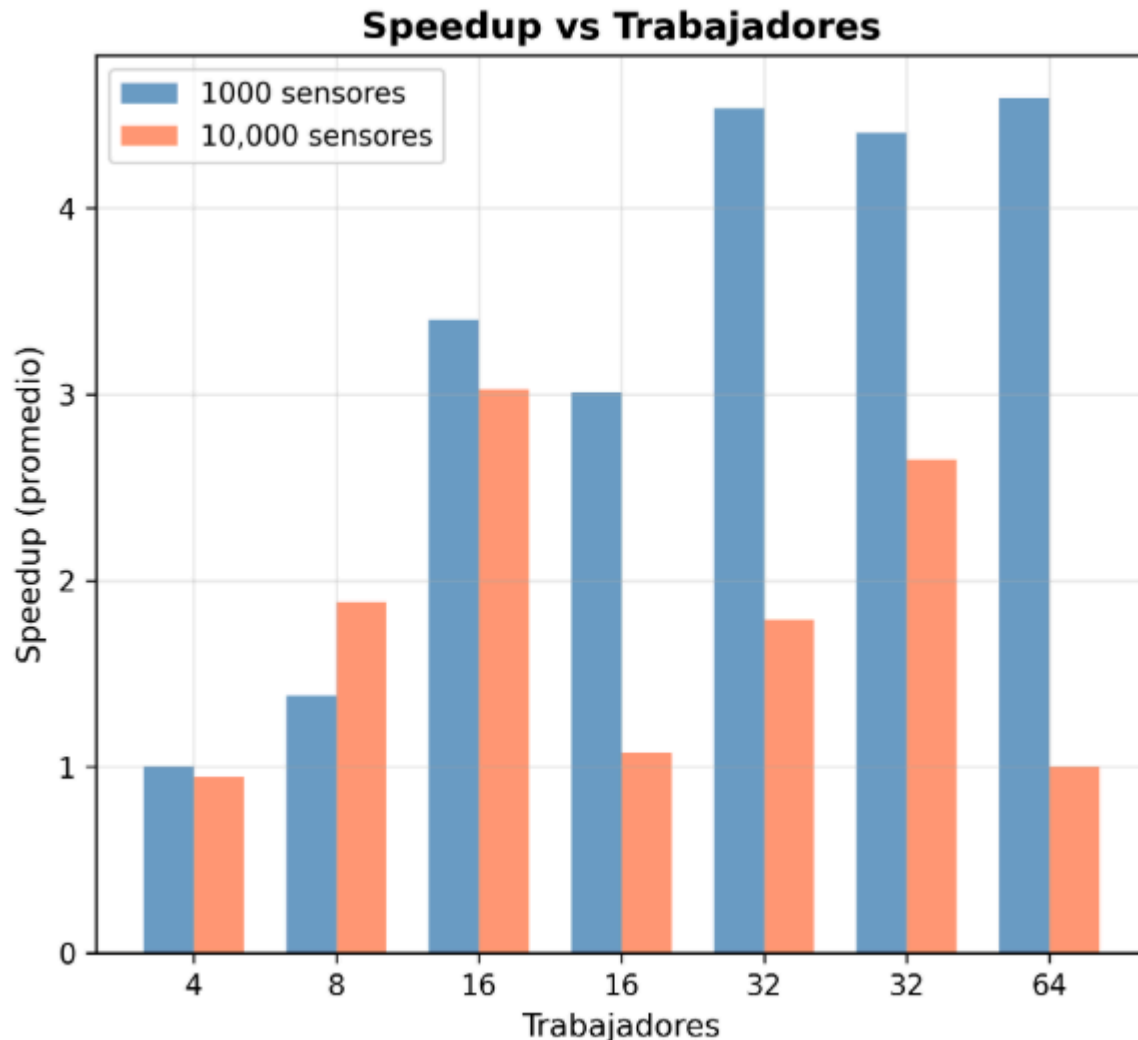
Descripción: Esta gráfica de barras compara el throughput (mensajes procesados por segundo) obtenido en cada configuración de procesos×hilos, mostrando lado a lado el promedio alcanzado con 1,000 sensores (azul) y con 10,000 sensores (naranja) para cada nivel de trabajadores totales (4, 8, 16, 32, 64).



Resultados obtenidos: El throughput aumenta de forma consistente conforme crece el número de trabajadores, tanto para 1,000 como para 10,000 sensores. La diferencia más notable ocurre en la configuración de 64 trabajadores (8×8), donde el escenario de 10,000 sensores alcanza 6,380 msg/seg frente a solo 1,000 msg/seg con 1,000 sensores, es decir, 6.4 veces más throughput. Esto sugiere que con una mayor carga de datos disponible, el sistema puede aprovechar mejor el paralelismo cuando hay suficientes trabajadores. En configuraciones intermedias (32 trabajadores) la mejora también es clara: 8×4 pasa de 961 a 1,764 msg/seg (1.84x) y 4×8 de 988 a 1,266 msg/seg (1.28x).

4.2 Speedup vs Trabajadores

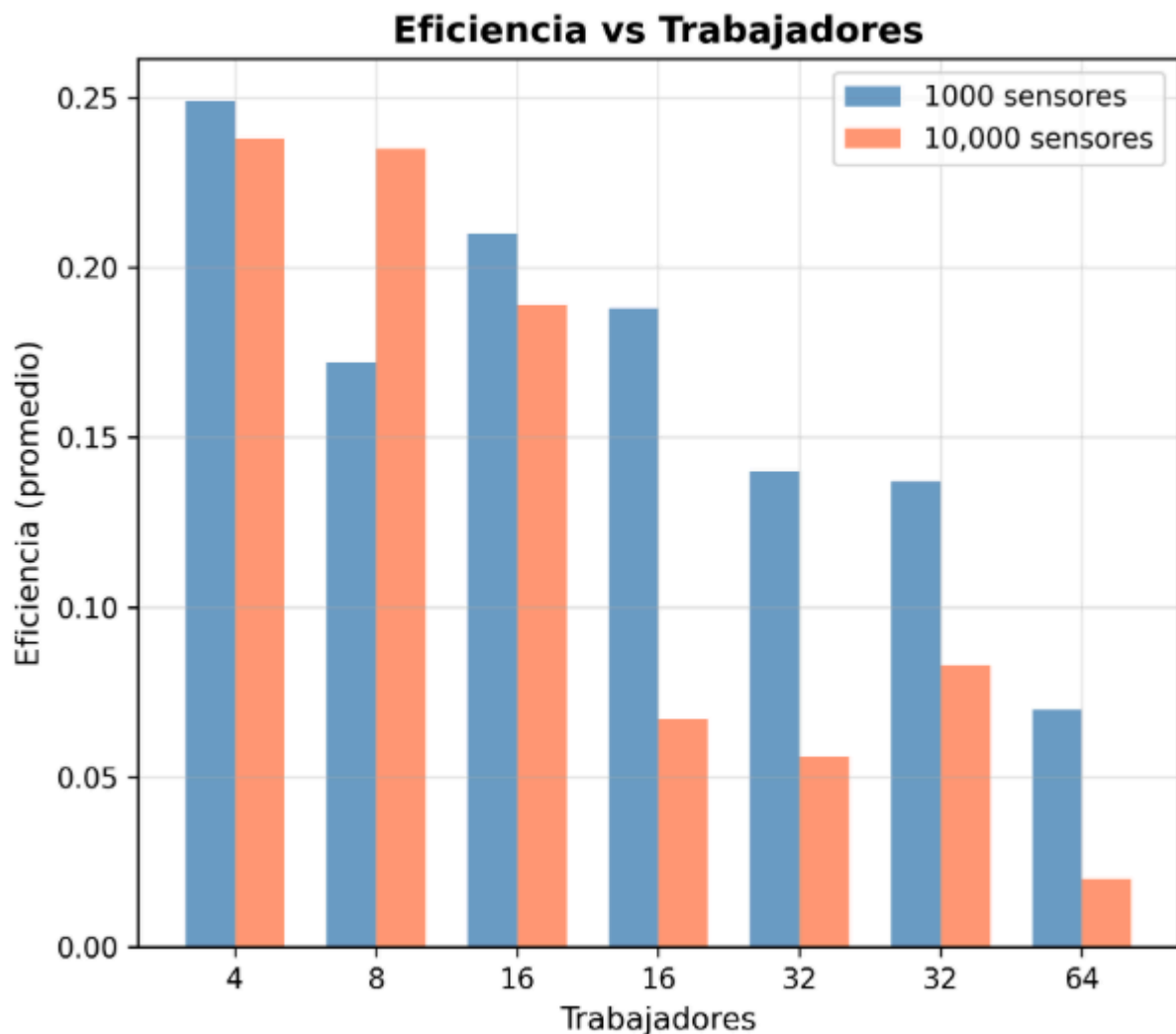
Descripción: Gráfica de barras que compara el speedup promedio (aceleración respecto a la configuración base) obtenido en cada nivel de trabajadores, contrastando ambos volúmenes de sensores.



Resultados obtenidos: Con 1,000 sensores el speedup crece de forma casi monótona con el número de trabajadores, llegando a 4.59x en la configuración 8×8. En cambio, con 10,000 sensores el speedup mejora hasta cierto punto (por ejemplo, 3.02x en 4×4 con 16 trabajadores) pero luego cae en las configuraciones más grandes, llegando a solo 1.00x en 8×8. Esto indica que, al aumentar la cantidad de sensores, agregar más trabajadores no siempre se traduce en mejor aceleración relativa; el overhead de coordinación entre procesos e hilos empieza a limitar la ganancia.

4.3 Eficiencia vs Trabajadores

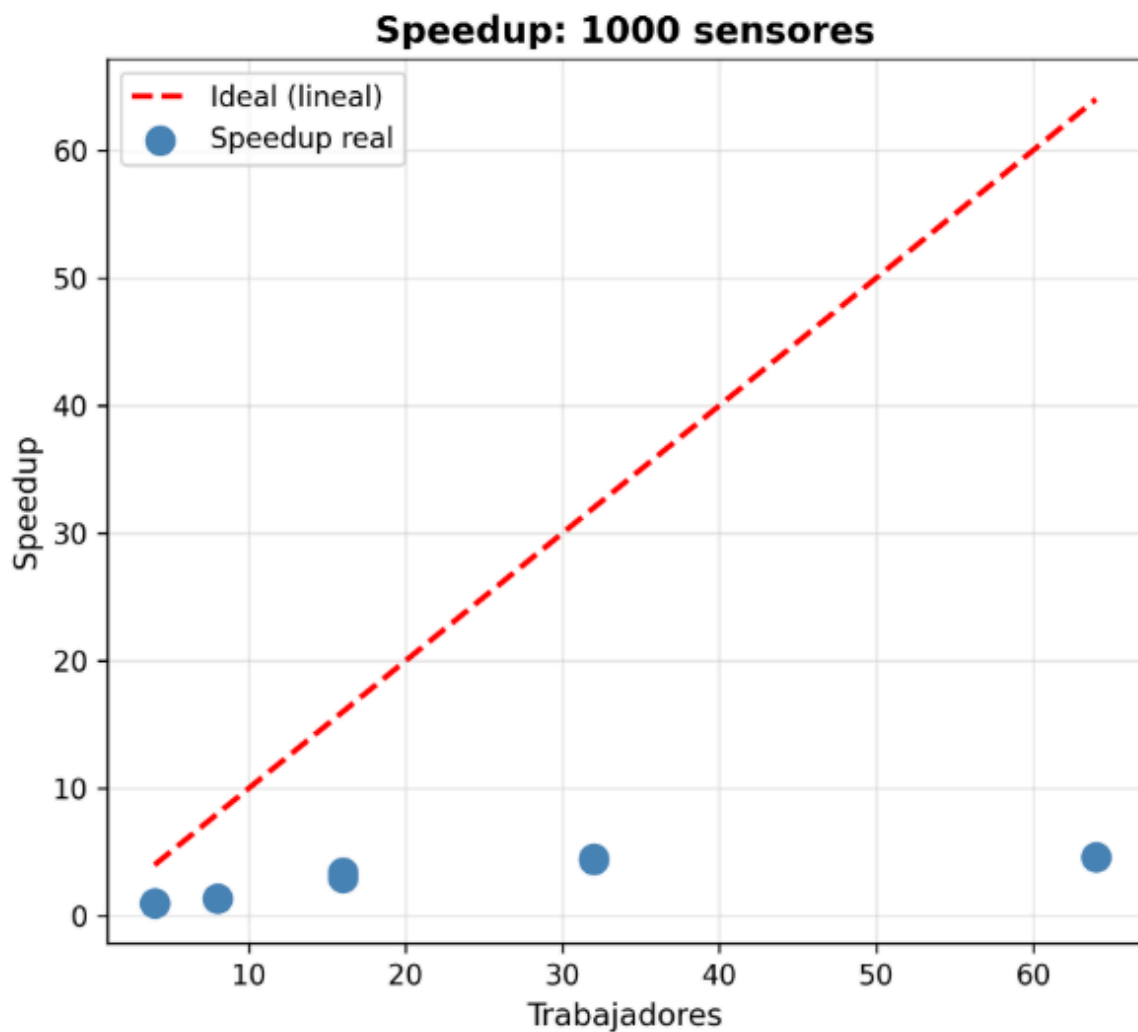
Descripción: Gráfica de barras que muestra la eficiencia promedio (speedup dividido entre el número de trabajadores) para cada configuración, comparando los dos volúmenes de datos.



Resultados obtenidos: La eficiencia disminuye conforme aumenta el número de trabajadores en ambos escenarios, lo cual es esperado ya que agregar más recursos rara vez escala linealmente. Sin embargo, la caída es mucho más pronunciada con 10,000 sensores: en la configuración 8×8 la eficiencia cae a solo 0.02, frente a 0.07 con 1,000 sensores. La configuración 2×4 (8 trabajadores) es la que mejor eficiencia mantiene en el escenario de 10,000 sensores (0.24), sugiriendo que menos trabajadores con mayor carga de datos por trabajador resulta más eficiente que repartir el trabajo entre muchos procesos/hilos.

4.4 Speedup ideal vs real (1,000 sensores)

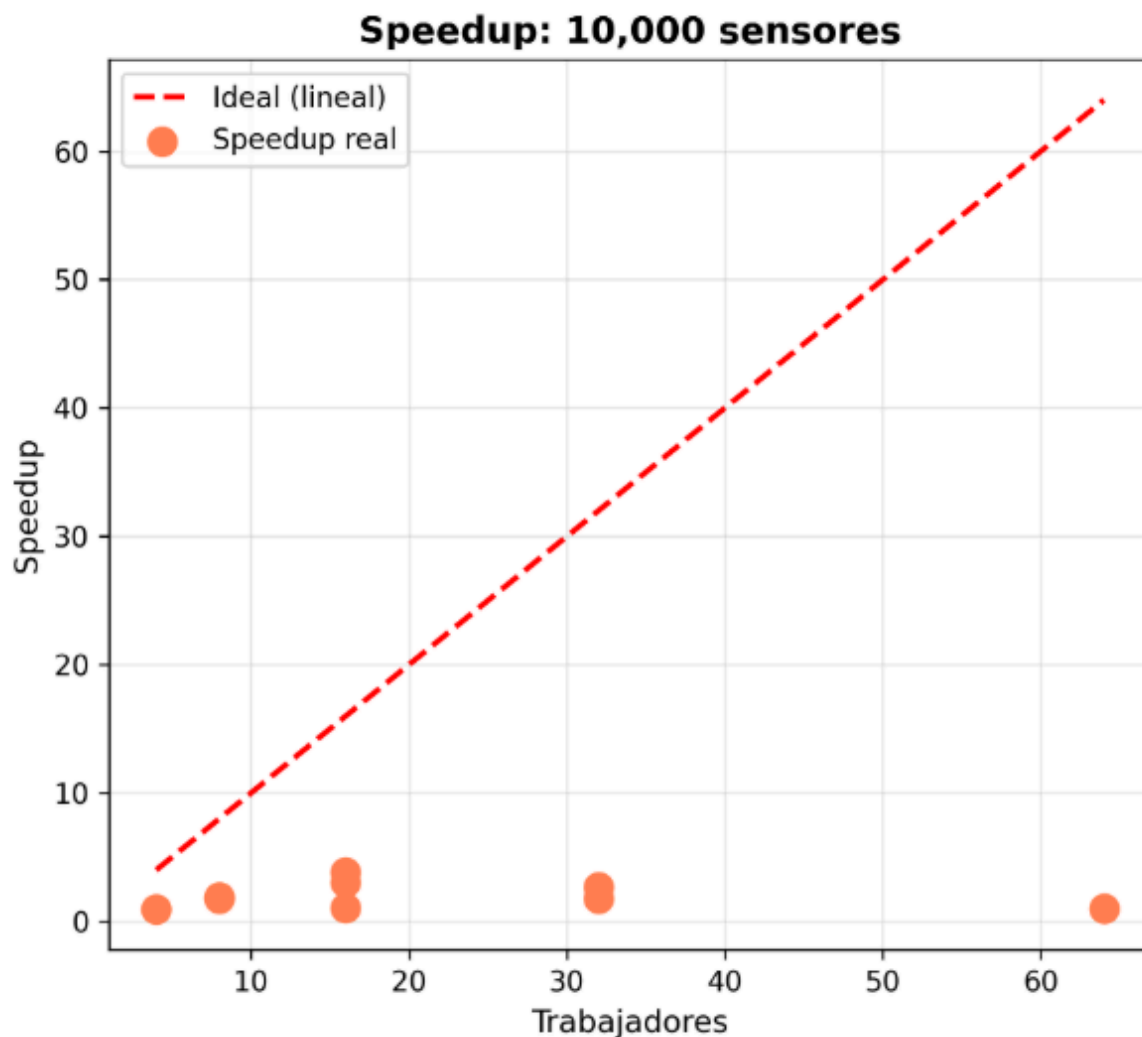
Descripción: Gráfica de dispersión y línea que compara el speedup ideal (lineal, igual al número de trabajadores) contra el speedup real obtenido con 1,000 sensores, en función del número de trabajadores.



Resultados obtenidos: El speedup real se aleja considerablemente del ideal lineal a partir de 16 trabajadores. Mientras que el speedup ideal en 64 trabajadores sería 64x, el real alcanzado fue de apenas 4.59x. Esto evidencia que el sistema no escala linealmente y que existe un límite práctico de paralelización, probablemente relacionado con el volumen de datos disponible (1,000 sensores) que no alcanza para saturar tantos trabajadores.

4.5 Speedup ideal vs real (10,000 sensores)

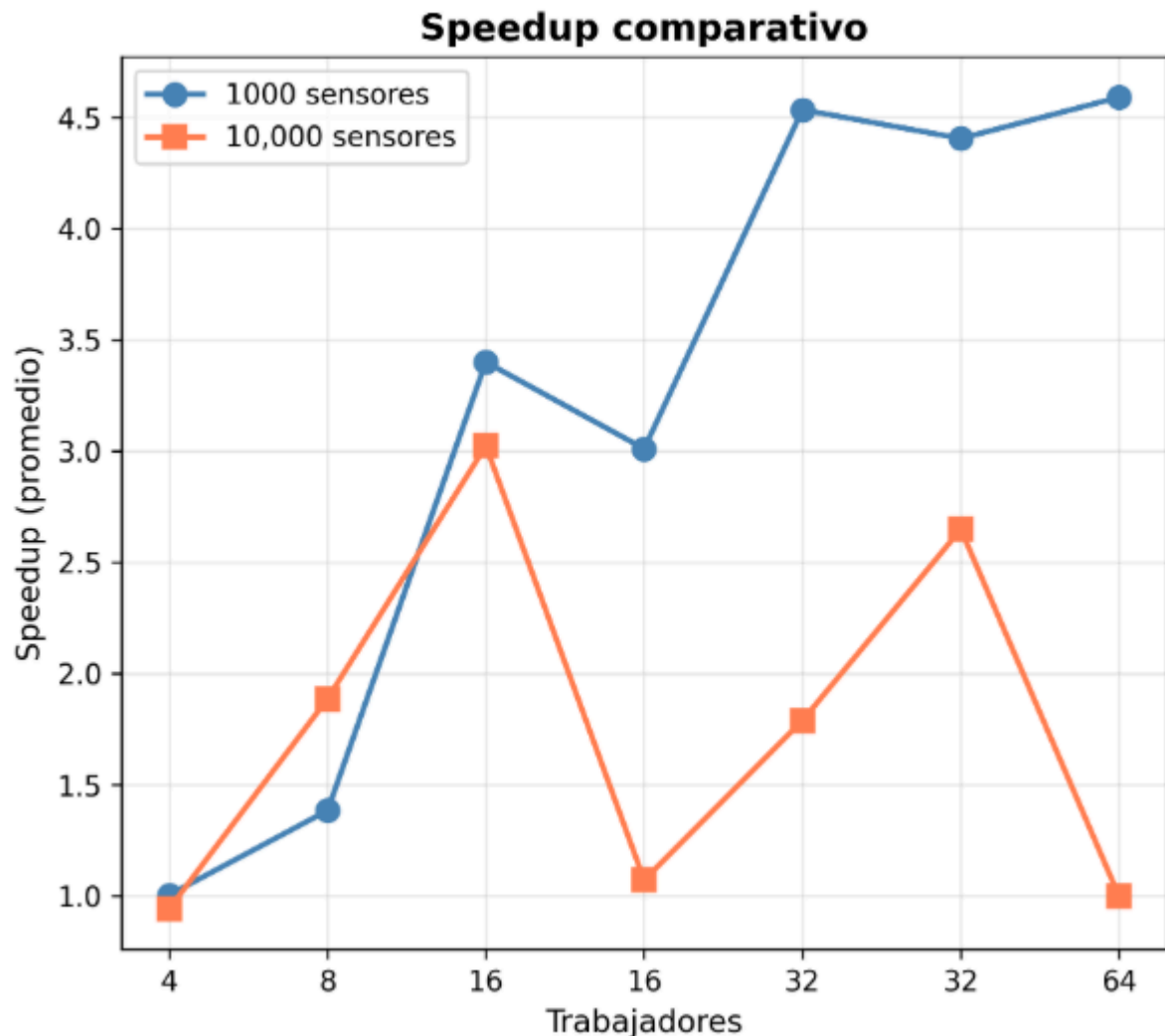
Descripción: Misma comparación que la gráfica anterior, pero aplicada al escenario de 10,000 sensores.



Resultados obtenidos: Al igual que con 1,000 sensores, el speedup real se separa del ideal, aunque el patrón es distinto: mejora hasta los 16 trabajadores (llegando a valores cercanos a $3x$ - $3.85x$) y después decae en 32 y 64 trabajadores. Esto refuerza la idea de que existe un punto óptimo de paralelización antes del cual agregar trabajadores ayuda, y después del cual el overhead de sincronización empieza a superar el beneficio, incluso con mayor volumen de datos.

4.6 Speedup comparativo (1,000 vs 10,000 sensores)

Descripción: Gráfica de líneas que superpone el speedup promedio de ambos escenarios en función del número de trabajadores, facilitando la comparación directa de las tendencias.



Resultados obtenidos: Las dos curvas muestran comportamientos claramente distintos: con 1,000 sensores el speedup crece de forma sostenida hasta los 64 trabajadores, mientras que con 10,000 sensores el speedup alcanza su punto máximo alrededor de los 16 trabajadores y luego desciende. Esto indica que el número óptimo de trabajadores depende directamente del volumen de datos a procesar: con más sensores, un número menor de trabajadores bien dimensionado resulta más efectivo que escalar agresivamente el paralelismo.

Gráficas:

<https://drive.google.com/drive/folders/1ncosrREYIoW15Do50T0JArWQyB9o6HUm?usp=sharing>

5. Comparativa de Resultados

Además de las pruebas por separado con 1000 y 10 000 sensores (secciones 3.4.1 y 3.4.2), también se hicieron pruebas en modo de repeticiones fijas, aislando el paralelismo por procesos MPI del paralelismo por hilos Pthreads (Tabla 5.1). Estos son los resultados obtenidos en conjunto.

Haciendo la comparación entre las tres corridas, podemos ver que aunque los valores absolutos no son comparables entre sí (el modo continuo mide mensajes/segundo con pausas de por medio, mientras que el modo repeticiones mide un trabajo fijo sin pausas, corriendo a máxima velocidad — por eso sus cifras son miles de veces más altas), el **patrón de Eficiencia se repite en las tres**: agregar hilos dentro de un mismo proceso pierde eficiencia más rápido que agregar procesos MPI. La Tabla 5.1 lo muestra de forma más clara porque aísla ambos mecanismos por separado (103.5% de eficiencia con 2 procesos puros, contra apenas 56.0% con 2 hilos puros); en las pruebas de 3.4.1 y 3.4.2, al combinar procesos e hilos en la mayoría de las configuraciones, esa diferencia se diluye un poco, pero la misma tendencia se asoma — por ejemplo, en 3.4.2, las configuraciones con más hilos por proceso (como 8×2) caen a una eficiencia mucho menor que las que reparten más por procesos con menos hilos cada uno.

Tabla 5.1: Experimento con 10 000 sensores (modo repeticiones fijas, MPI puro vs. Pthreads puro)

Procesos	Hilos	Trabajadores	Mensajes totales	Tiempo (s)	Throughput (msgs/seg)	Speedup	Eficiencia
1	1	1	200,000	0.247331	808,633.12	1.00x	100%
1	2	2	200,000	0.228677	874,597.12	1.08x	54%
1	4	4	200,000	0.268501	744,874.87	0.92x	23%
1	8	8	200,000	0.209497	954,666.39	1.18x	15%
2	1	2	200,000	0.136474	1,465,480.68	1.81x	90%

En las tres pruebas se observa además el mismo comportamiento de rendimientos decrecientes: la Eficiencia baja de forma sostenida conforme crecen los trabajadores, sin importar el volumen de sensores simulados ni el modo de medición usado. Esto refuerza que el cuello de botella principal del sistema no está en la generación de datos en sí (que escala bien al agregar procesos MPI), sino en la concurrencia dentro de un mismo proceso — ya sea por contención de memoria compartida entre hilos, o por la cantidad de conexiones MQTT independientes compitiendo dentro del mismo espacio de proceso. Al aparecer de forma consistente en tres mediciones hechas con metodologías distintas, esta conclusión queda mejor respaldada que si viniera de una sola corrida de datos.

6. Conclusiones

- Se implementó un sistema de simulación paralela de sensores IoT que integra MPI, Pthreads y MQTT en una arquitectura desacoplada, donde el simulador y el frontend se comunican únicamente a través de un broker, sin conocerse entre sí.
- La distribución geográfica por MPI y la concurrencia por Pthreads permitieron paralelizar la generación de datos, pero no aportaron por igual: el paralelismo

por procesos escaló casi al ideal, mientras que el paralelismo por hilos tuvo un impacto mucho más limitado.

- Hallazgo principal: aislando ambos mecanismos, 2 procesos $\times$ 1 hilo alcanzó 2.07x de speedup con 103.5% de eficiencia — prácticamente lineal — mientras que escalar solo con hilos (2, 4, 8) dio apenas 1.01x-1.13x, con la eficiencia cayendo hasta 12.7%. El mismo patrón se repitió, más diluido, en las corridas con 1000 y 10 000 sensores: a igual cantidad de trabajadores, más procesos y menos hilos rindió mejor que al revés.
- La eficiencia cae de forma sostenida al aumentar el paralelismo, sobre todo al agregar hilos — coherente con contención de memoria compartida y múltiples conexiones MQTT compitiendo dentro de un mismo proceso.
- El rendimiento se estabiliza en configuraciones con muchos trabajadores, apuntando a cuellos de botella en el broker y en la concurrencia interna del proceso, más que en la generación de datos.
- El proyecto valida la programación paralela híbrida como solución viable para simuladores IoT, con una lección de diseño concreta: en esta arquitectura conviene priorizar procesos MPI sobre hilos al escalar. Queda como trabajo futuro confirmar si la causa es la contención dentro del proceso o un límite del broker.